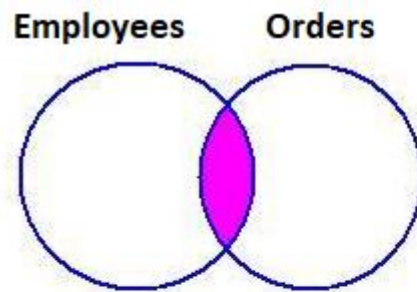


Databases

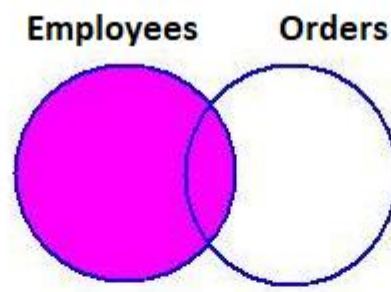
Course 8

Joins, Union, Intersect, Minus – Oracle SQL implementation

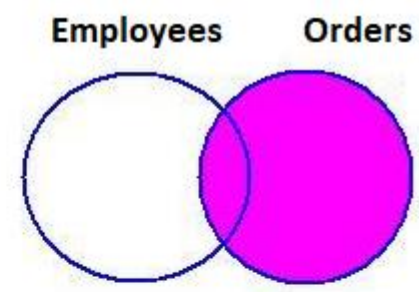
Types of joins



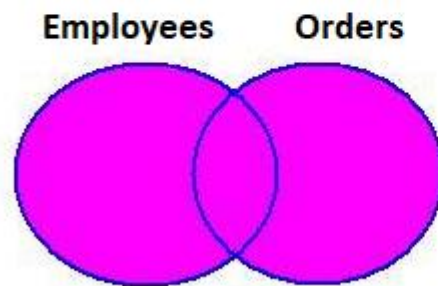
Inner join



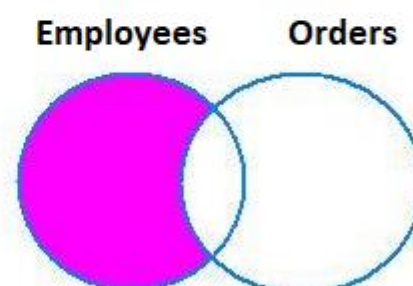
Left outer join



Right outer join



Full outer join



Minus

Inner join and natural join

- $R_1 \bowtie R_2$
- $JOIN(R_1, R_2, condition)$
- $JOIN(R_1, R_2, condition) = RESTRICT(PRODUCT(R_1, R_2), condition)$

Inner join

- An operation defined on two relations, R_1 and R_2 , through which a new relation is constructed, R_3 , whose schema is obtained by the reunion of attributes from the initial relations and its extension contains the tuples obtained by concatenating the tuples of R_1 with the tuples of R_2 having the same values for the attributes used in the join condition

Inner Join

Employees

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	142	Curtis	Davies
2	143	Randall	Matos
3	144	Peter	Vargas
4	155	Oliver	Tuvault
5	156	Janette	King



Orders

ORDER_ID	ORDER_DATE	SALES_REP_ID	CUSTOMER_ID
2360	11/14/2004		107
2359	1/8/2003		106
2362	11/13/2004		109
2358	1/8/2002	155	105
2407	6/29/2004	155	165
2449	6/13/2004	155	146
2444	7/27/2004	155	109
2354	7/14/2002	155	104
2411	5/24/2004	156	169
2410	5/24/2002	156	168
2440	8/31/2004	156	107
2391	2/27/2003	156	153
2381	5/14/2002	156	106

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DATE	SALES_REP_ID
1	155	Oliver	Tuvault	2449	6/13/2004	155
2	155	Oliver	Tuvault	2444	7/27/2004	155
3	155	Oliver	Tuvault	2354	7/14/2002	155
4	155	Oliver	Tuvault	2358	1/8/2002	155
5	155	Oliver	Tuvault	2407	6/29/2004	155
6	156	Janette	King	2381	5/14/2002	156
7	156	Janette	King	2440	8/31/2004	156
8	156	Janette	King	2391	2/27/2003	156
9	156	Janette	King	2410	5/24/2002	156
10	156	Janette	King	2411	5/24/2004	156

SQL Examples

- **select * from EMPLOYEES t, ORDERS o where t.employee_id=o.sales_rep_id;**
- **select * from EMPLOYEES t join ORDERS o on t.employee_id=o.sales_rep_id;**

Natural join

- Assume the existence of attributes with the same name in both relationships
- **select * from EMPLOYEES t natural join ORDERS o**

11235 rows selected in 10.428 seconds

- **select * from CUSTOMERS t natural join ORDERS o**

105 rows selected in 1.132 seconds

Columns of CUSTOMERS			Columns of ORDERS		
	Name	Type		Name	Type
1	CUSTOMER_ID	NUMBER(6)	1	ORDER_ID	NUMBER(12)
2	CUST_FIRST_NAME	VARCHAR2(20)	2	ORDER_DATE	DATE
3	CUST_LAST_NAME	VARCHAR2(20)	3	ORDER_MODE	VARCHAR2(8)
4	ADDRESS	VARCHAR2(30)	4	CUSTOMER_ID	NUMBER(6)
5	POSTAL_CODE	VARCHAR2(30)	5	ORDER_STATUS	NUMBER(2)
6	CITY	VARCHAR2(30)	6	ORDER_TOTAL	NUMBER(8,2)
7	STATE	VARCHAR2(30)	7	SALES_REP_ID	NUMBER(6)
8	COUNTRY	VARCHAR2(30)	8	PROMOTION_ID	NUMBER(6)

Natural Join vs Inner Join

```
select * from employees natural join departments;
```

32 rows selected in 0.072 seconds

```
select * from employees e join departments d on  
e.department_id=d.department_id;
```

106 rows selected in 0.110 seconds

Columns of EMPLOYEES		
	Name	Type
1	EMPLOYEE_ID	NUMBER(6)
2	FIRST_NAME	VARCHAR2(20)
3	LAST_NAME	VARCHAR2(25)
4	EMAIL	VARCHAR2(25)
5	PHONE_NUMBER	VARCHAR2(20)
6	HIRE_DATE	DATE
7	JOB_ID	VARCHAR2(10)
8	SALARY	NUMBER(8,2)
9	COMMISSION_PCT	NUMBER(2,2)
10	MANAGER_ID	NUMBER(6)
11	DEPARTMENT_ID	NUMBER(4)

Columns of DEPARTMENTS		
	Name	Type
1	DEPARTMENT_ID	NUMBER(4)
2	DEPARTMENT_NAME	VARCHAR2(30)
3	MANAGER_ID	NUMBER(6)

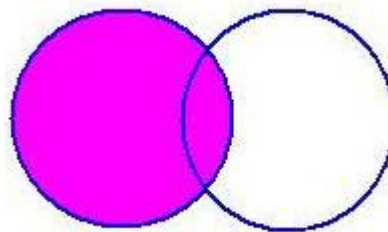
Outer Join

- Operation by which from two relations, R_1 and R_2 , a new relation, R_3 , results through the junction of relations R_1 and R_2 , to which the tuples of R_1 and/or R_2 that have not participated in the join are added.
- These tuples are filled in R_3 with NULL values for the corresponding relationship attributes (R_2 and R_1 respectively).
- $R_1 \underline{\bowtie} R_2$
- *EXT-JOIN* (R_1, R_2)

Left outer Join

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	142	Curtis	Davies
2	143	Randall	Matos
3	144	Peter	Vargas
4	155	Oliver	Tuvault
5	156	Janette	King

Employees Orders



Left outer join

ORDER_ID	ORDER_DATE	SALES_REP_ID	CUSTOMER_ID
2360	11/14/2004		107
2359	1/8/2003		106
2362	11/13/2004		109
2358	1/8/2002	155	105
2407	6/29/2004	155	165
2449	6/13/2004	155	146
2444	7/27/2004	155	109
2354	7/14/2002	155	104
2411	5/24/2004	156	169
2410	5/24/2002	156	168
2440	8/31/2004	156	107
2391	2/27/2003	156	153
2381	5/14/2002	156	106

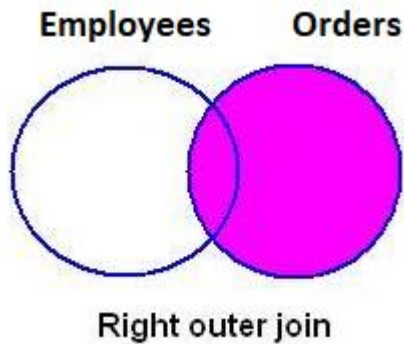
1	150	Peter	Tucker			
2	151	David	Bernstein			
3	152	Peter	Hall			
4	153	Christopher	Olsen			
5	154	Nanette	Cambrault			
6	155	Oliver	Tuvault	2449	6/13/2004	155
7	155	Oliver	Tuvault	2444	7/27/2004	155
8	155	Oliver	Tuvault	2407	6/29/2004	155
9	155	Oliver	Tuvault	2358	1/8/2002	155
10	155	Oliver	Tuvault	2354	7/14/2002	155
11	156	Janette	King	2411	5/24/2004	156
12	156	Janette	King	2410	5/24/2002	156
13	156	Janette	King	2391	2/27/2003	156
14	156	Janette	King	2440	8/31/2004	156
15	156	Janette	King	2381	5/14/2002	156
16	157	Patrick	Sully			
17	158	Allan	McEwen			

SQL Examples

- **select * from EMPLOYEES t, ORDERS o where t.employee_id=o.sales_rep_id(+);**
- **select * from EMPLOYEES t left join ORDERS o on t.employee_id=o.sales_rep_id;**

Right outer join

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME
1	142	Curtis	Davies
2	143	Randall	Matos
3	144	Peter	Vargas
4	155	Oliver	Tuvault
5	156	Janette	King



ORDER_ID	ORDER_DATE	SALES_REP_ID	CUSTOMER_ID
2360	11/14/2004		107
2359	1/8/2003		106
2362	11/13/2004		109
2358	1/8/2002	155	105
2407	6/29/2004	155	165
2449	6/13/2004	155	146
2444	7/27/2004	155	109
2354	7/14/2002	155	104
2411	5/24/2004	156	169
2410	5/24/2002	156	168
2440	8/31/2004	156	107
2391	2/27/2003	156	153
2381	5/14/2002	156	106

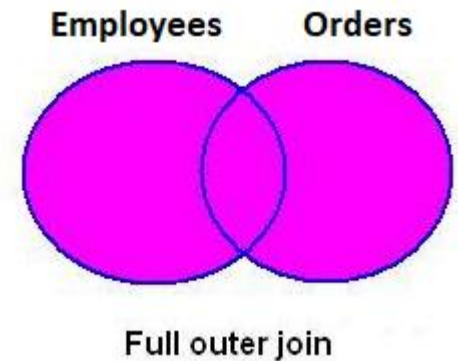
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DATE	SALES_REP_ID
1	155	Oliver	Tuvault	2449	6/13/2004	155
2	155	Oliver	Tuvault	2444	7/27/2004	155
3	155	Oliver	Tuvault	2407	6/29/2004	155
4	155	Oliver	Tuvault	2358	1/8/2002	155
5	155	Oliver	Tuvault	2354	7/14/2002	155
6	156	Janette	King	2411	5/24/2004	156
7	156	Janette	King	2410	5/24/2002	156
8	156	Janette	King	2391	2/27/2003	156
9	156	Janette	King	2440	8/31/2004	156
10	156	Janette	King	2381	5/14/2002	156
11				2428	11/10/2004	
12				2426	11/17/2003	
13				2421	3/12/2004	
14				2390	11/18/2004	
15				2389	6/4/2002	

SQL Examples

- **select * from EMPLOYEES t, ORDERS o where
t.employee_id(+) = o.sales_rep_id;**
- **select * from EMPLOYEES t right join ORDERS o on
t.employee_id = o.sales_rep_id;**

Full Outer Join

1	150	Peter	...	Tucker	...			
2	151	David	...	Bernstein	...			
3	152	Peter	...	Hall	...			
4	153	Christopher	...	Olsen	...			
5	154	Nanette	...	Cambrault	...			
6	155	Oliver	...	Tuvault	...	2449	6/13/2004	155
7	155	Oliver	...	Tuvault	...	2444	7/27/2004	155
8	155	Oliver	...	Tuvault	...	2407	6/29/2004	155
9	155	Oliver	...	Tuvault	...	2358	1/8/2002	155
10	155	Oliver	...	Tuvault	...	2354	7/14/2002	155
11	156	Janette	...	King	...	2411	5/24/2004	156
12	156	Janette	...	King	...	2410	5/24/2002	156
13	156	Janette	...	King	...	2391	2/27/2003	156
14	156	Janette	...	King	...	2440	8/31/2004	156
15	156	Janette	...	King	...	2381	5/14/2002	156
16	157	Patrick	...	Sully	...			
17	158	Allan	...	McEwen	...			
18			...		...	2428	11/10/2004	
19			...		...	2426	11/17/2003	
20			...		...	2421	3/12/2004	
21			...		...	2390	11/18/2004	
22			...		...	2389	6/4/2002	
23			...		...	2388	6/4/2004	



SQL Examples

- **select * from EMPLOYEES t, ORDERS o where
t.employee_id=o.sales_rep_id(+) union**
**select * from EMPLOYEES t, ORDERS o where
t.employee_id(+)=o.sales_rep_id;**
- **select * from EMPLOYEES t full join ORDERS o on
t.employee_id=o.sales_rep_id;**

(Other) Types of Join

- Equijoin
- Cartesian Product
- Non-equijoin
- Outer Join
- Self Join
- Semijoin
- Antijoin

Equijoin (Inner Join)

Oracle proprietary join

=

Ex: select sales_rep_id, order_id,
cust_first_name, cust_last_name
from customers c, orders o
where c.customer_id=o.customer_id;

ANSI/ISO SQL99 compliant standard join

NATURAL JOIN/JOIN+USING/JOIN+ON

- NATURAL JOIN (**one** field called the same way – same data type, in both tables)

Ex: *select sales_rep_id, order_id, cust_first_name, cust_last_name from customers **natural join** orders;*

- JOIN+ON (no restrictions)

*select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c **join** orders o **on** c.customer_id=o.customer_id;*

- JOIN+USING (for tables that have fields called the same way)

! The column that is used in USING cannot have a qualifier (alias or table name) anywhere in the select

*select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c **join** orders o **using** (customer_id);*

Cartesian Product

Oracle proprietary join	ANSI/ISO SQL99 compliant standard join
<pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c, orders o;</pre>	<pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c cross join orders o;</pre>

Non-equijoin

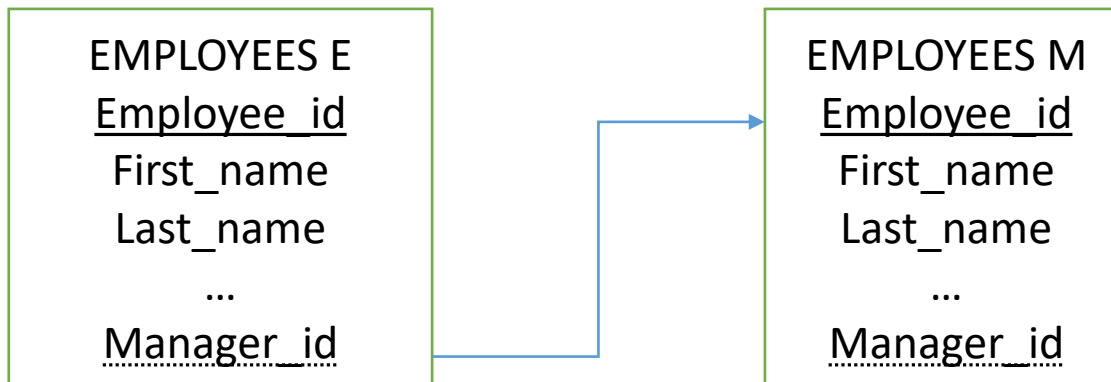
Oracle proprietary join	ANSI/ISO SQL99 compliant standard join
<pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c, orders o where c.customer_id<o.customer_id;</pre>	<pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c join orders o on c.customer_id<o.customer_id;</pre>

Outer Join

Oracle proprietary join	ANSI/ISO SQL99 compliant standard join
<pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c, orders o where c.customer_id=o.customer_id(+);</pre> <pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c, orders o where o.customer_id(+)=c.customer_id;</pre>	LEFT/RIGHT/FULL OUTER JOIN <pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from customers c left join orders o on c.customer_id=o.customer_id;</pre> <pre>select sales_rep_id, order_id, cust_first_name, cust_last_name from orders o right join customers c on c.customer_id=o.customer_id;</pre>

Self Join

Oracle proprietary join	ANSI/ISO SQL99 compliant standard join
<pre>select e.employee_id, e.first_name, e.manager_id, m.first_name from employees e, employees m where e.manager_id=m.employee_id;</pre>	<pre>select e.employee_id, e.first_name, e.manager_id, m.first_name from employees e join employees m on e.manager_id=m.employee_id;</pre>



Semi Join

- A semijoin returns rows that match an EXISTS subquery without duplicating rows from the left side of the predicate when multiple rows on the right side satisfy the criteria of the subquery.
- Semijoin and antijoin transformation cannot be done if the subquery is on an OR branch of the WHERE clause.
- Ex:

```
SELECT * FROM departments
```

```
WHERE EXISTS
```

```
(SELECT * FROM employees
```

```
WHERE departments.department_id = employees.department_id
```

```
AND employees.salary > 2500)
```

```
ORDER BY department_name;
```

Anti Join

- An antijoin returns rows from the left side of the predicate for which there are no corresponding rows on the right side of the predicate. It returns rows that fail to match (NOT IN) the subquery on the right side.

- Ex:

```
SELECT * FROM employees
WHERE department_id NOT IN
(SELECT department_id FROM departments
WHERE location_id = 1700)
ORDER BY last_name;
```

Union

- It is an operator defined on two relations, R1 and R2, both with the same scheme. The union operator has the function to build a new relation R3, with the same scheme as R1 and R2, but with the tuples from R1 and R2, taken together once.
- $R_1 \cup R_2$
- *OR* (R_1, R_2)
- *APPEND* (R_1, R_2)
- *UNION* (R_1, R_2)

UNION Example

```
select t.first_name from
EMPLOYEES t order by 1
```

	FIRST_NAME
1	Adam
2	Alana
3	Alberto
4	Alexander
5	Alexander
6	Alexis
7	Allan
8	Alyssa
9	Amit
10	Anthony

```
select c.cust_first_name from CUSTOMERS c
union
select e.first_name from employees e
order by 1
```

	CUST_FIRST_NAME
1	Adam
2	Ajay
3	Alain
4	Alan
5	Alana
6	Albert
7	Alberto
8	Alec
9	Alexander
10	Alexis

SQL

Output Statistics

```
select c.cust_first_name from
CUSTOMERS c order by 1
```

	CUST_FIRST_NAME
1	Ajay
2	Ajay
3	Alain
4	Alain
5	Alain
6	Alan
7	Alan
8	Albert
9	Albert
10	Albert
11	Alec

```
select c.cust_first_name,c.cust_last_name from CUSTOMERS c
union
select e.first_name,e.last_name from EMPLOYEES e
order by 1
```

	CUST_FIRST_NAME	CUST_LAST_NAME
1	Adam	Fripp
2	Ajay	Andrews
3	Ajay	Sen
4	Alain	Barkin
5	Alain	Dreyfuss
6	Alain	Siegel
7	Alan	Hunter
8	Alan	Minnelli
9	Alana	Walsh
10	Albert	Bel Geddes

UNION ALL Example

```
select t.first_name from
EMPLOYEES t order by 1
```

	FIRST_NAME
1	Adam
2	Alana
3	Alberto
4	Alexander
5	Alexander
6	Alexis
7	Allan
8	Alyssa
9	Amit
10	Anthony

```
select c.cust_first_name from CUSTOMERS c
union all
select e.first_name from EMPLOYEES e
order by 1
```

	CUST_FIRST_NAME
1	Adam
2	Ajay
3	Ajay
4	Alain
5	Alain
6	Alain
7	Alan
8	Alan
9	Alana
10	Albert

SQL	Output	Statistics
<pre>select c.cust_first_name from CUSTOMERS c order by 1</pre>		
	CUST_FIRST_NAME	
1	Ajay	...
2	Ajay	...
3	Alain	...
4	Alain	...
5	Alain	...
6	Alan	...
7	Alan	...
8	Albert	...
9	Albert	...
10	Albert	...
11	Alec	...

```
select c.cust_first_name,c.cust_last_name from CUSTOMERS c
union all
select e.first_name,e.last_name from EMPLOYEES e
order by 1
```

	CUST_FIRST_NAME	CUST_LAST_NAME
1	Adam	Fripp
2	Ajay	Andrews
3	Ajay	Sen
4	Alain	Barkin
5	Alain	Siegel
6	Alain	Dreyfuss
7	Alan	Minnelli
8	Alan	Hunter
9	Alana	Walsh
10	Albert	Spacek
11	Albert	Bel Geddes

UNION ALL Example

SQL		Output	Statistics
<pre>select first_name,count(*) no from (select t.first_name first_name from EMPLOYEES t union all select c.cust_first_name from CUSTOMERS c) group by first_name order by 2 desc</pre>			
            			
		FIRST_NAME	NO
▶	1	Alexander	5
	2	John	4
	3	Daniel	4
	4	Bruce	4
	5	Kevin	4
	6	Michael	3
	7	Keir	3
	8	Peter	3
	9	Ellen	3
	10	Bryan	3
	11	Rosanne	3
	12	Marlon	3
	13	Donald	3
	14	Kris	3
	15	Matthias	3

Minus

- It is an operator defined on two relations, R1 and R2, both with the same scheme. The minus operator has the function to construct a new relation R3 on the basis of relations R1 and R2, with the same scheme, but with the extension of those tuples with the relationship R1 not found in the relation R2.
- $R_1 - R_2$
- *REMOVE* (R_1, R_2)
- *MINUS* (R_1, R_2)

Minus

```
select * from orders;
```

	ORDER_ID	ORDER_DATE	ORDER_M	CUSTO
1	2458	8/16/2004	direct	101
2	2397	11/19/2004	direct	102
3	2454	10/2/2004	direct	103
4	2354	7/14/2002	direct	104
5	2358	1/8/2002	direct	105
6	2381	5/14/2002	direct	106
7	2440	8/31/2004	direct	107
8	2357	1/8/2003	direct	108
9	2394	2/10/2002	direct	109
10	2435	9/2/2004	direct	144
11	2455	9/20/2004	direct	145
12	2379	5/16/2004	direct	146
13	2396	2/2/2003	direct	147

SQL

Output Statistics

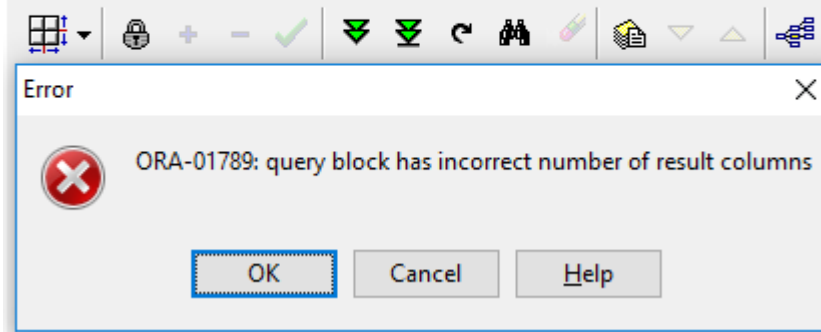
```
select * from CUSTOMERS c
order by 1
```

	CUSTO	CUST_FIRST_NAME	CUST_LAST_NAME
1	101	Constantin	Welles
2	102	Harrison	Pacino
3	103	Manisha	Taylor
4	104	Harrison	Sutherland
5	105	Matthias	MacGraw
6	106	Matthias	Hannah
7	107	Matthias	Cruise
8	108	Meenakshi	Mason
9	109	Christian	Cage
10	110	Charlie	Sutherland
11	111	Charlie	Pacino

SQL

Output Statistics

```
select * from CUSTOMERS
minus
select * from ORDERS;
```



Minus

SQL		Output	Statistics
<pre>select customer_id from CUSTOMERS minus select customer_id from ORDERS order by 1</pre>			
		CUSTOMER_ID	
▶	1	110	
	2	111	
	3	112	
	4	113	
	5	114	
	6	115	
	7	124	
	8	125	
	9	126	
	10	127	
	11	128	
	12	129	
	13	130	
	14	131	

SQL

Output

Statistics

```
select * from CUSTOMERS where customer_id in(
select customer_id  from CUSTOMERS
minus
select customer_id  from ORDERS)
order by 1;
```

		CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME
▶	1	110	Charlie	Sutherland
	2	111	Charlie	Pacino
	3	112	Guillaume	Jackson
	4	113	Daniel	Costner
	5	114	Dianne	Derek
	6	115	Geraldine	Schneider
	7	124	Diane	Mason
	8	125	Dianne	Andrews
	9	126	Charles	Field
	10	127	Charles	Broderick
	11	128	Isabella	Reed
	12	129	Louis	Jackson
	13	130	Louis	Edwards
	14	131	Doris	Dutt

Minus

```
select c.* from CUSTOMERS c left join ORDERS o
on c.customer_id=o.customer_id
where o.customer_id is null
order by 1
```

		CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME	
▶	1	110	Charlie	Sutherland	...
	2	111	Charlie	Pacino	...
	3	112	Guillaume	Jackson	...
	4	113	Daniel	Costner	...
	5	114	Dianne	Derek	...
	6	115	Geraldine	Schneider	...
	7	124	Diane	Mason	...
	8	125	Dianne	Andrews	...
	9	126	Charles	Field	...
	10	127	Charles	Broderick	...
	11	128	Isabella	Reed	...
	12	129	Louis	Jackson	...
	13	130	Louis	Edwards	...
	14	131	Doris	Dutt	...

```
select c.* from CUSTOMERS c left join ORDERS o
on c.customer_id=o.customer_id
minus
select c.* from CUSTOMERS c join ORDERS o
on c.customer_id=o.customer_id
order by 1
```

		CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME	
▶	1	110	Charlie	Sutherland	...
	2	111	Charlie	Pacino	...
	3	112	Guillaume	Jackson	...
	4	113	Daniel	Costner	...
	5	114	Dianne	Derek	...
	6	115	Geraldine	Schneider	...
	7	124	Diane	Mason	...
	8	125	Dianne	Andrews	...
	9	126	Charles	Field	...
	10	127	Charles	Broderick	...
	11	128	Isabella	Reed	...
	12	129	Louis	Jackson	...
	13	130	Louis	Edwards	...
	14	131	Doris	Dutt	...

Intersection

- It is an operator defined on two relations, R_1 and R_2 both with the same scheme, which consists in building a new relation, R_3 , with a scheme identical to the operands and with an extension that contains the tuples that are in R_1 and R_2 .
- $R_1 \cap R_2$
- *INTERSECT* (R_1, R_2)
- *AND* (R_1, R_2)

Intersect

SQL

Output

Statistics

```
select * from CUSTOMERS where customer_id in(  
select customer_id  from CUSTOMERS  
intersect  
select customer_id  from ORDERS)  
order by 1;
```



























		CUSTOMER_ID	CUST_FIRST_NAME		CUST_LAST_NAME	
▶	1	101	Constantin	...	Welles	...
	2	102	Harrison	...	Pacino	...
	3	103	Manisha	...	Taylor	...
	4	104	Harrison	...	Sutherland	...
	5	105	Matthias	...	MacGraw	...
	6	106	Matthias	...	Hannah	...
	7	107	Matthias	...	Cruise	...
	8	108	Meenakshi	...	Mason	...
	9	109	Christian	...	Cage	...
	10	116	Geraldine	...	Martin	...
	11	117	Guillaume	...	Edwards	...
	12	118	Maurice	...	Mahoney	...
	13	119	Maurice	...	Hasan	...
	14	120	Diane	...	Higgins	...

Intersect

```
select c.* from CUSTOMERS c join ORDERS o
on c.customer_id=o.customer_id
```

	CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME
1	101	Constantin	Welles
2	101	Constantin	Welles
3	101	Constantin	Welles
4	101	Constantin	Welles
5	102	Harrison	Pacino
6	102	Harrison	Pacino
7	102	Harrison	Pacino
8	102	Harrison	Pacino
9	103	Manisha	Taylor
10	103	Manisha	Taylor
11	103	Manisha	Taylor
12	103	Manisha	Taylor
13	104	Harrison	Sutherland
14	104	Harrison	Sutherland
15	104	Harrison	Sutherland
16	104	Harrison	Sutherland
17	105	Matthias	MacGraw
18	105	Matthias	MacGraw
19	105	Matthias	MacGraw
20	105	Matthias	MacGraw
21	106	Matthias	Hannah
22	106	Matthias	Hannah
23	106	Matthias	Hannah
24	106	Matthias	Hannah
25	107	Matthias	Cruise

```
select distinct c.* from CUSTOMERS c join ORDERS o
on c.customer_id=o.customer_id order by 1
```

	CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME	AD
1	101	Constantin	Welles	514
2	102	Harrison	Pacino	251
3	103	Manisha	Taylor	876
4	104	Harrison	Sutherland	644
5	105	Matthias	MacGraw	401
6	106	Matthias	Hannah	160
7	107	Matthias	Cruise	239
8	108	Meenakshi	Mason	136
9	109	Christian	Cage	190
10	116	Geraldine	Martin	Luc
11	117	Guillaume	Edwards	180
12	118	Maurice	Mahoney	492
13	119	Maurice	Hasan	331
14	120	Diane	Higgins	113

Intersect

SQL	Output	Statistics
<pre>select c.* from CUSTOMERS c left join ORDERS o on c.customer_id=o.customer_id intersect select c.* from CUSTOMERS c right join ORDERS o on c.customer_id=o.customer_id order by 1</pre>		

Intersect

SQL

Output

Statistics

```
select c.*  from CUSTOMERS c left join ORDERS o
on c.customer_id=o.customer_id
intersect
select c.*  from CUSTOMERS c right join ORDERS o
on c.customer_id=o.customer_id order by 1
```

		CUSTOMER_ID	CUST_FIRST_NAME	CUST_LAST_NAME
▶ 1	101	Constantin	...	Welles
2	102	Harrison	...	Pacino
3	103	Manisha	...	Taylor
4	104	Harrison	...	Sutherland
5	105	Matthias	...	MacGraw
6	106	Matthias	...	Hannah
7	107	Matthias	...	Cruise
8	108	Meenakshi	...	Mason
9	109	Christian	...	Cage
10	116	Geraldine	...	Martin
11	117	Guillaume	...	Edwards
12	118	Maurice	...	Mahoney
13	119	Maurice	...	Hasan
14	120	Diane	...	Higgins
15	121	Dianne	...	Sen
16	122	Maurice	...	Daltrey

What does this display?

```
select  
t.employee_id,t.first_name,t.last_name,  
t.salary,t.department_id from EMPLOYEES t  
where department_id<>80 or salary>7000  
order by 5
```

What does this display?

SQL

Output

Statistics

```
select t.employee_id,t.first_name,t.last_name,  
t.salary,t.department_id from EMPLOYEES t  
where department_id<>80 or salary>7000  
order by 5
```

		EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
52		194	Samuel	McCain	3200.00	50
53		195	Vance	Jones	2800.00	50
54		196	Alana	Walsh	3100.00	50
55		197	John	FEENEY	3000.00	50
56		107	Diana	Lorentz	4200.00	60
57		106	Valli	Pataballa	4800.00	60
58		105	David	Austin	4800.00	60
59		104	Bruce	Ernst	6000.00	60
60		103	Alexander	Hunold	9000.00	60
61		204	Hermann	Baer	10000.00	70
62		149	Eleni	Zlotkey	10500.00	80
63		176	Jonathon	Taylor	8600.00	80
64		175	Alyssa	Hutton	8800.00	80
65		174	Ellen	Abel	11000.00	80
66		172	Elizabeth	Bates	7300.00	80
67		171	William	Smith	7400.00	80
68		170	Taylor	Fox	9600.00	80
69		169	Harrison	Bloom	10000.00	80
70		168	Lisa	Ozer	11500.00	80
71		164	Mattea	Marvins	7200.00	80

What does this display?

SQL

Output

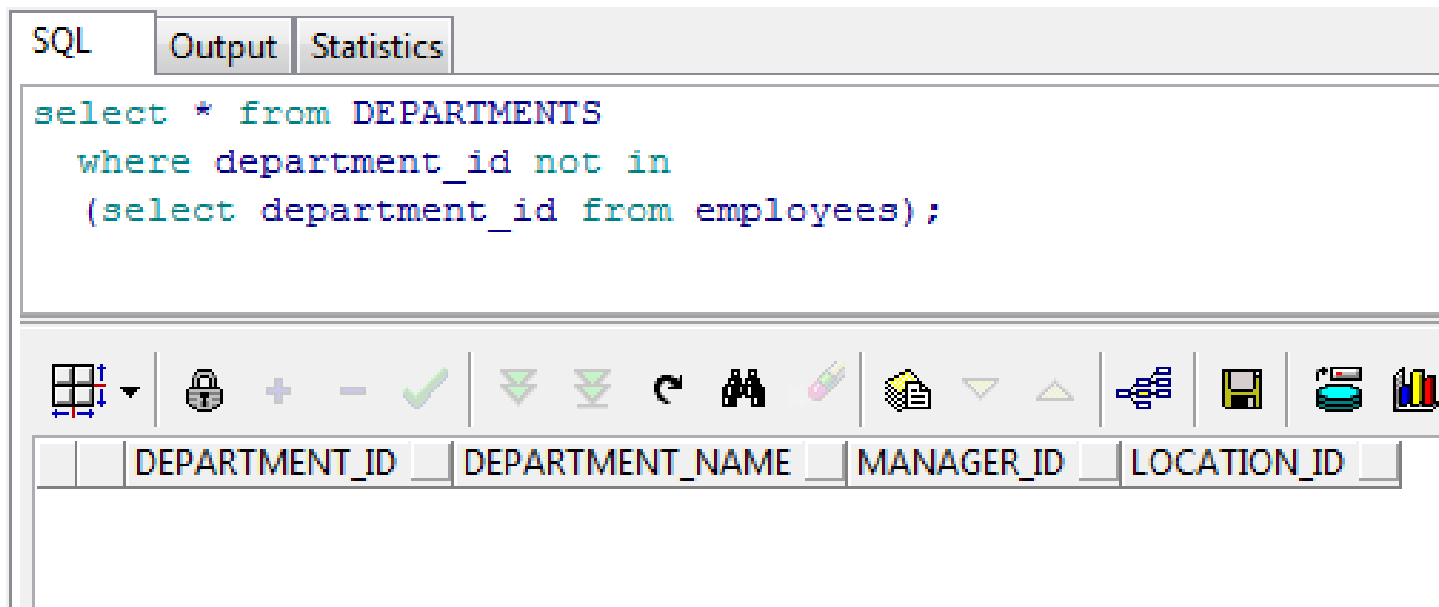
Statistics

```
select * from DEPARTMENTS
  where department_id in
    (select department_id from employees);
```

		DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
▶	1	10	Administration	200	1700
	2	20	Marketing	201	1800
	3	30	Purchasing	114	1700
	4	40	Human Resources	203	2400
	5	50	Shipping	121	1500
	6	60	IT	103	1400
	7	70	Public Relations	204	2700
	8	80	Sales	145	2500
	9	90	Executive	100	1700
	10	100	Finance	108	1700
	11	110	Accounting	205	1700

What does this display?



The screenshot shows an SQL IDE interface with three tabs: "SQL", "Output", and "Statistics". The "SQL" tab is active, displaying the following query:

```
select * from DEPARTMENTS
  where department_id not in
    (select department_id from employees);
```

Below the query editor is a toolbar with various icons for editing and executing SQL. Below the toolbar is a table with the following columns:

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
---------------	-----------------	------------	-------------

What does this display?

SQL Output Statistics

```
select * from DEPARTMENTS
where department_id not in
(select department_id from employees);
```

DEPARTMENT_ID DEPARTMENT_NAME MANAGER_ID LOCATION_ID

```
select department_id from employees;
```

	DEPARTMENT_ID
73	80
74	80
75	80
76	80
77	80
78	80
79	80
80	80
81	80
82	80
83	80
84	80
85	80
86	80
87	80
88	
89	80
90	50
91	50

What does this display?

SQL

Output

Statistics

```
select * from DEPARTMENTS
  where department_id not in
    (select department_id from employees
     where department_id is not null);|
```

		DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
▶	1	120	Treasury	...	1700
	2	130	Corporate Tax	...	1700
	3	140	Control And Credit	...	1700
	4	150	Shareholder Services	...	1700
	5	160	Benefits	...	1700
	6	170	Manufacturing	...	1700
	7	180	Construction	...	1700
	8	190	Contracting	...	1700
	9	200	Operations	...	1700
	10	210	IT Support	...	1700
	11	220	NOC	...	1700
	12	230	IT Helpdesk	...	1700
	13	240	Government Sales	...	1700
	14	250	Retail Sales	...	1700
	15	260	Recruiting	...	1700
	16	270	Payroll	...	1700

What does this display?

```
select * from EMPLOYEES e, DEPARTMENTS t
where e.department_id<>t.department_id
```

What does this display?

```
select * from EMPLOYEES e, DEPARTMENTS t
where e.department_id<>t.department_id
```

	EMPLOYEE_ID	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	198	50	Donald	10	Administration
2	199	50	Douglas	10	Administration
3	201	20	Michael	10	Administration
4	202	20	Pat	10	Administration
5	203	40	Susan	10	Administration
6	204	70	Hermann	10	Administration
7	205	110	Shelley	10	Administration
8	206	110	William	10	Administration
9	100	90	Steven	10	Administration
10	101	90	Neena	10	Administration
11	102	90	Lex	10	Administration
12	103	60	Alexander	10	Administration
13	104	60	Bruce	10	Administration
14	105	60	David	10	Administration
15	106	60	Valli	10	Administration
16	107	60	Diana	10	Administration
17	108	100	Nancy	10	Administration
18	109	100	Daniel	10	Administration
19	110	100	John	10	Administration
20	111	100	Ismael	10	Administration
21	112	100	Jose Manuel	10	Administration
22	113	100	Luis	10	Administration
23	114	30	Den	10	Administration
24	115	30	Alexander	10	Administration
25	116	30	Shelli	10	Administration
26	117	30	Sigal	10	Administration
27	118	30	Guy	10	Administration
28	119	30	Karen	10	Administration

References

- Ramez Elmasri, Shamkant B. Navathe, *Fundamentals of database systems*, 6th ed, Addison-Wesley Publishing House, 2011, ISBN-13: 978-0-136-08620-8 (Cap 17,18,19)
- Ion Lungu, Adela Bâra, Constanța Bodea, Iuliana Botha, Vlad Diaconița, Alexandra Florea, Anda Velicanu, *Tratat de baze de date. Organizare, proiectare și implementare*, Editura ASE(Cap 5.3)